

MARK HANEY

STAFF BACKEND SOFTWARE ENGINEER | NODE.JS TECHNICAL LEAD | MICROSERVICES | CLOUD

Dardanelle, AR

<https://www.linkedin.com/in/markhaney-33b675ba>

+1 (430) 279-2231 | markhaney.dev@outlook.com

PROFESSIONAL SUMMARY

Staff Software Engineer specializing in **Node.js backend engineering** for SaaS, GovTech, PropTech, nonprofit CRM, and cloud service industries, with strong ownership across **NestJS 8–10**, **Express.js 4–5**, **TypeScript**, **PostgreSQL**, **MongoDB**, **Redis**, **AWS**, **Docker**, **Kubernetes**, and CI/CD delivery. I Experienced in building secure APIs, event-driven services, multi-tenant platforms, background jobs, observability, and production incident resolution while improving performance, reducing defects, modernizing legacy services, and mentoring teams through clean architecture, testing, and scalable backend practices.

TECH SKILLS

- ❖ **Backend:** Node.js 16–22, NestJS 8–10, Express.js 4–5, REST APIs, GraphQL, WebSockets, Microservices, Event-Driven Architecture
- ❖ **Languages:** TypeScript 4–5, JavaScript ES6+, SQL, Bash
- ❖ **Databases:** PostgreSQL, MongoDB, MySQL, Redis, Elasticsearch
- ❖ **Cloud & DevOps:** AWS, ECS, Lambda, S3, SQS, SNS, CloudWatch, Docker, Kubernetes, Terraform, GitHub Actions, GitLab CI
- ❖ **Testing:** Jest, Mocha, Chai, Supertest, Pact, Unit Testing, Integration Testing, API Contract Testing
- ❖ **Architecture:** Multi-Tenant SaaS, CQRS, Domain-Driven Design, API Gateway, Background Jobs, Queue Processing, Caching, Observability

PROFESSIONAL EXPERIENCE

Staff Software Engineer

Relatio

Jun 2022 – Apr 2026

Las Vegas, NV

- ❖ Designed **Node.js 18–22** backend services with **NestJS 9–10**, **TypeScript 4–5**, **PostgreSQL 14–16**, and **Redis 6–7** for a multi-tenant SaaS platform that required secure data isolation, faster reporting workflows, and stable API performance under large customer workloads.
- ❖ Led the migration from legacy **Express.js 4** modules into modular **NestJS 9** services, improving dependency boundaries, reducing duplicate business logic, and making future backend feature delivery easier for multiple product teams.
- ❖ Implemented a role-based access control feature using **JWT**, scoped permissions, tenant-aware middleware, and audit logs, reducing authorization-related production issues while supporting enterprise customer security reviews.
- ❖ Resolved recurring timeout bugs in customer analytics endpoints by rewriting heavy aggregation queries, adding **Redis caching**, and introducing pagination contracts that improved average response time by **42%** during peak usage.
- ❖ Built event-driven background processing with **AWS SQS**, **BullMQ**, and retry-safe workers, reducing failed async jobs and making invoice, notification, and report generation more reliable.

- ❖ Improved API observability with **OpenTelemetry**, structured logs, request correlation IDs, and CloudWatch dashboards, helping the team identify slow dependencies before they became customer-facing incidents.
- ❖ Refactored shared validation layers with **class-validator**, reusable DTOs, and centralized error handling, reducing inconsistent API error responses and making frontend integration more predictable.
- ❖ Partnered with frontend engineers to define stable REST contracts for React dashboards, using **OpenAPI** documentation and integration tests to reduce regressions during weekly releases.
- ❖ Hardened production services by adding rate limits, request size controls, secure headers, and dependency scanning, improving backend security posture without slowing normal customer traffic.
- ❖ Optimized CI/CD pipelines with **GitHub Actions**, Docker layer caching, and parallel Jest test execution, reducing backend deployment time by **29%** across staging and production environments.
- ❖ Mentored engineers on **Node.js memory profiling**, async error handling, and clean service boundaries, improving code review quality and reducing repeated architecture mistakes across squads.
- ❖ Supported production incidents through root-cause analysis, rollback planning, hotfix validation, and postmortems, consistently turning operational issues into backlog improvements instead of temporary fixes.

Senior Software Engineer

AppFolio

Feb 2018 – May 2022

Goleta, CA

- ❖ Built scalable backend APIs with **Node.js 12–16**, **Express.js 4**, **TypeScript**, **PostgreSQL**, and **Redis** for property management workflows involving payments, lease data, tenant communication, and internal operations.
- ❖ Modernized older JavaScript services into typed **TypeScript 4** modules, reducing runtime type-related defects and improving maintainability for high-volume backend workflows.
- ❖ Implemented a payment notification service using **AWS SNS**, **SQS**, idempotency keys, and retry policies, reducing duplicate event processing during high-volume billing cycles.
- ❖ Solved intermittent database lock issues by analyzing slow queries, adding targeted indexes, and splitting large transactional updates into safer batches that improved job completion time by **38%**.
- ❖ Migrated containerized services from manual deployment scripts to **Docker** and **AWS ECS**, reducing release failures and giving engineers a more predictable promotion path.
- ❖ Improved unit and integration test coverage with **Jest**, **Supertest**, and seeded test databases, helping the team catch contract breaks before staging deployments.
- ❖ Built backend support for tenant messaging features, including queue-based delivery, webhook retries, and delivery-status tracking for operational transparency.
- ❖ Collaborated with product managers to break complex leasing workflows into incremental backend releases, allowing safer delivery without large risky deployments.
- ❖ Handled production debugging for memory leaks, unhandled promise rejections, and slow endpoints by using heap snapshots, structured logs, and controlled rollback procedures.

Software Engineer

Blackbaud

Jul 2016 – Jan 2018

Charleston, SC

- ❖ Developed nonprofit CRM backend services using **Node.js 6–10**, **Express.js 4**, **MongoDB**, **PostgreSQL**, and REST APIs for donor management, reporting, and campaign workflows.

- ❖ Reworked legacy callback-heavy JavaScript into Promise-based service layers, reducing async control-flow bugs and making production issues easier to trace.
- ❖ Built secure integration endpoints for donor imports, payment status updates, and campaign activity synchronization, using validation rules to prevent malformed customer data from entering core systems.
- ❖ Fixed recurring CSV import failures caused by encoding mismatches, oversized payloads, and partial validation logic by adding streaming parsers, row-level errors, and retry-safe processing.
- ❖ Improved MongoDB query performance by adding compound indexes and restructuring filters, reducing report-generation latency by **36%** for larger nonprofit accounts.
- ❖ Added automated test coverage with **Mocha**, **Chai**, and API integration tests, reducing regression bugs across release cycles involving shared backend modules.
- ❖ Supported migration of selected backend jobs from older cron scripts into queue-based workers, improving reliability for nightly sync and reporting tasks.
- ❖ Partnered with QA and support teams to investigate customer-reported defects, reproduce edge cases, and ship targeted fixes with clear rollback notes.
- ❖ Documented API behavior, failure scenarios, and deployment steps so new engineers could support services without depending on tribal knowledge.

Software Developer

Granicus

May 2014 – Jun 2016

Dallas, TX

- ❖ Built backend features for GovTech communication platforms using **Node.js 0.10–6**, **Express.js 3–4**, **MongoDB**, **PostgreSQL**, and RESTful services supporting public-sector publishing workflows.
- ❖ Implemented API endpoints for content scheduling, subscriber preferences, and notification delivery, helping agencies manage digital communication with fewer manual operations.
- ❖ Migrated selected services from older **Express.js 3** routing patterns to **Express.js 4** middleware structure, improving maintainability and reducing routing defects by **23%**.
- ❖ Resolved production issues around duplicate notifications by adding idempotency checks, better queue state tracking, and defensive validation before message dispatch.
- ❖ Optimized database reads for public content feeds by introducing query indexes and lightweight caching, improving endpoint response times under higher traffic.
- ❖ Created backend utilities for audit trails, admin activity logs, and permission checks to support compliance-focused public-sector workflows.
- ❖ Improved deployment reliability by updating build scripts, environment configuration handling, and smoke-test checks before releases reached production.
- ❖ Worked closely with senior engineers to review code, debug API issues, and learn scalable backend patterns across older JavaScript service architectures.

Junior Software Developer

Vology

Jun 2013 – Apr 2014

Gainesville, FL

- ❖ Supported internal service tools using **Node.js**, **Express.js 3**, **MySQL**, **MongoDB**, and JavaScript to automate inventory, ticketing, and operational reporting workflows.
- ❖ Built small REST endpoints and backend scripts that reduced manual spreadsheet updates for support and operations teams handling recurring service requests.

- ❖ Fixed data mismatch bugs between internal tools by improving validation rules, normalizing request payloads, and adding clearer error messages for failed records.
- ❖ Assisted with migration of legacy utility scripts into reusable Node modules, making common database access and logging patterns easier to maintain.
- ❖ Wrote unit tests, deployment notes, and troubleshooting documentation while learning production support discipline from senior engineers and operations stakeholders.

EDUCATION

Arkansas State University

Bachelor's degree, Computer Science (Sep 2009 – May 2013)